

Software modelling

Software Modelling is the creation of tangible, but abstract, representations of a system so that you can communicate your design ideas, critique them and explore viable alternatives.

Software Design

Software Design is all about purposefully choosing the behaviour and structure of your software system.

- System behaviour: how your systems responds to inputs and events
- System structure: how parts of the system collaborate to achieve the goals of the system

UML use-case diagram

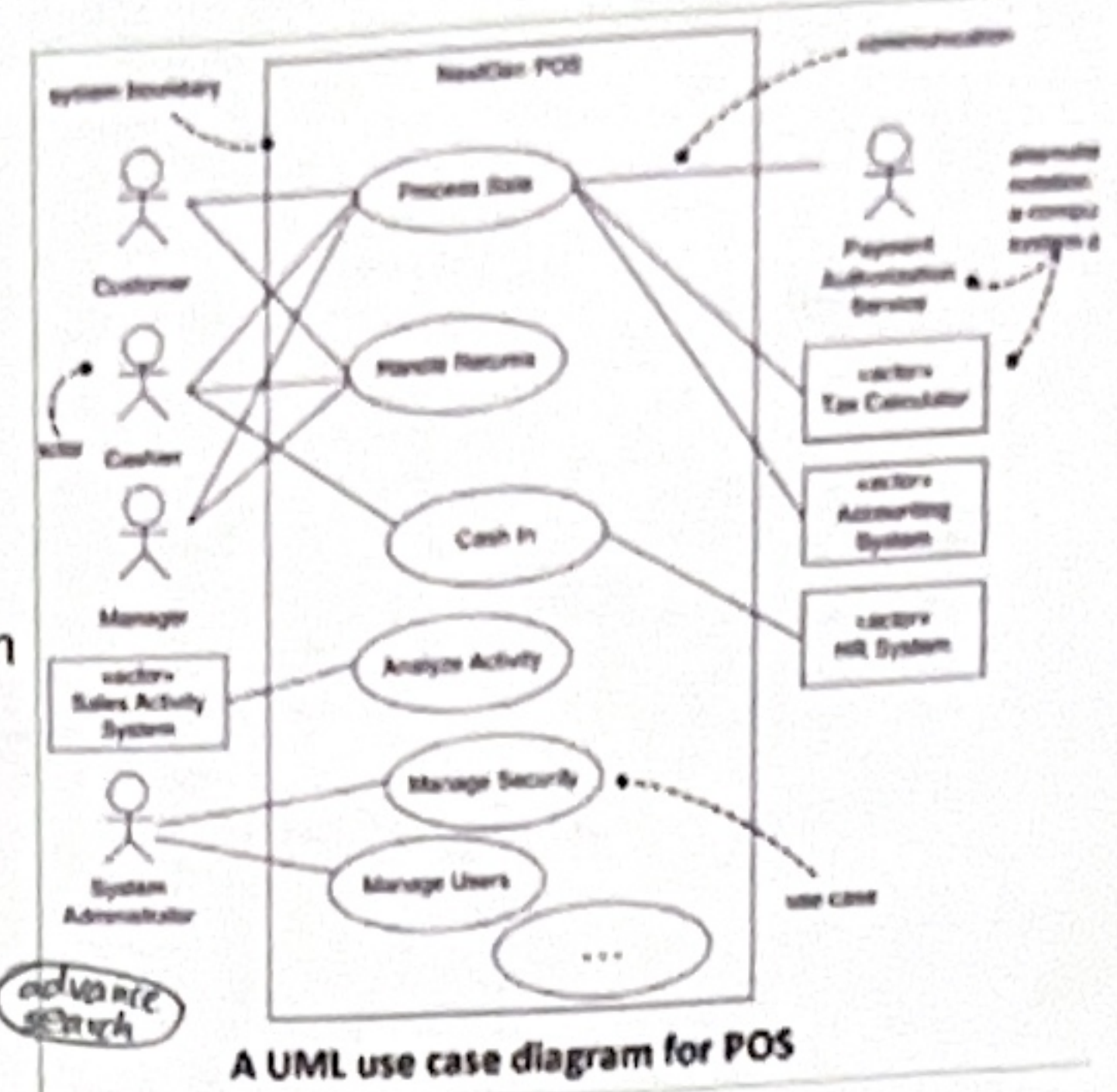
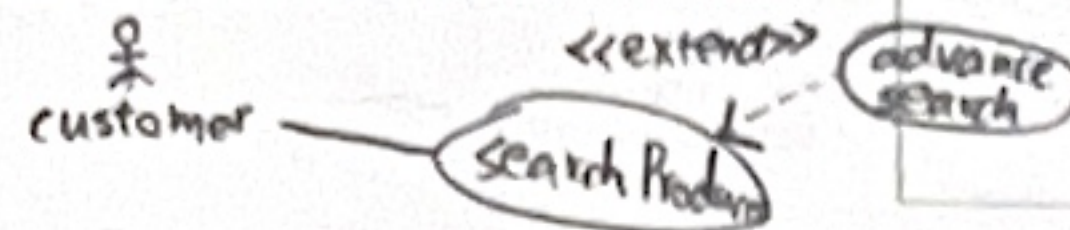
A UML use case diagram gives a context diagram of a system and its environment

- Showing the names of use cases, actors and their relationships

Object-oriented Analysis

is a process of creating a description of the domain from the perspective of object.

This means analyse use cases and identify objects / concepts in the problem domain. The concepts and behaviours are captured in the form of Domain Models and Sequence Diagrams.



A UML use case diagram for POS

Object-Oriented Software Design

is a process of creating a conceptual solution by defining software objects and their collaboration. We model the conceptual solution using Design Class Diagram & Design Sequence Diagram.

ObjectOriented Domain Models

Analysis: An investigation of the problem & requirements.

- Object-oriented software analysis emphasises finding and describing objects and concepts in the problem domain.

ObjectOriented Design Models

Design: A conceptual solution to a problem that meets the requirements.

- Object-oriented software design emphasises defining software objects and their collaboration.

ObjectOriented Implementation

- Implementation: A concrete solution to a problem that meets the requirements.
- Object-oriented software implementation: implementation in objectoriented languages and technologies.

Responsibility-Driven Design (RDD)

RDD focuses on assigning responsibility to software objects

Two types of responsibility:

- **Knowing responsibilities** include:
 - knowing about private encapsulated data
 - knowing about related objects
 - knowing about things it can derive or calculate

- **Doing responsibilities** include:
 - doing something itself, such as creating an object or doing a calculation
 - initiating action in other objects
 - controlling and coordinating activities in other objects

Software Architecture

concerns on a highlevel organization of software elements in the system and how the elements collaborate to meets high impact system requirements, supports the business logic, and is straightforward to maintain and evolve.

Architectural Analysis

is an activity of analyzing requirements (especially the nonfunctional requirements) to identify the factors that can have an impact on the design, hence reduce risk of missing a critical factor in the design of a system.

Logical Architecture (LA)

The largescale organisation of the software classes into packages, subsystems and layers

- UML Package diagram illustrate the logical architecture
- The logical architecture can organize software classes in layers (i.e., Layered Architecture)
- UML Component diagram provide a highlevel implementation view of the architecture which consider the external services (e.g., databases)

Unified Process (UP)

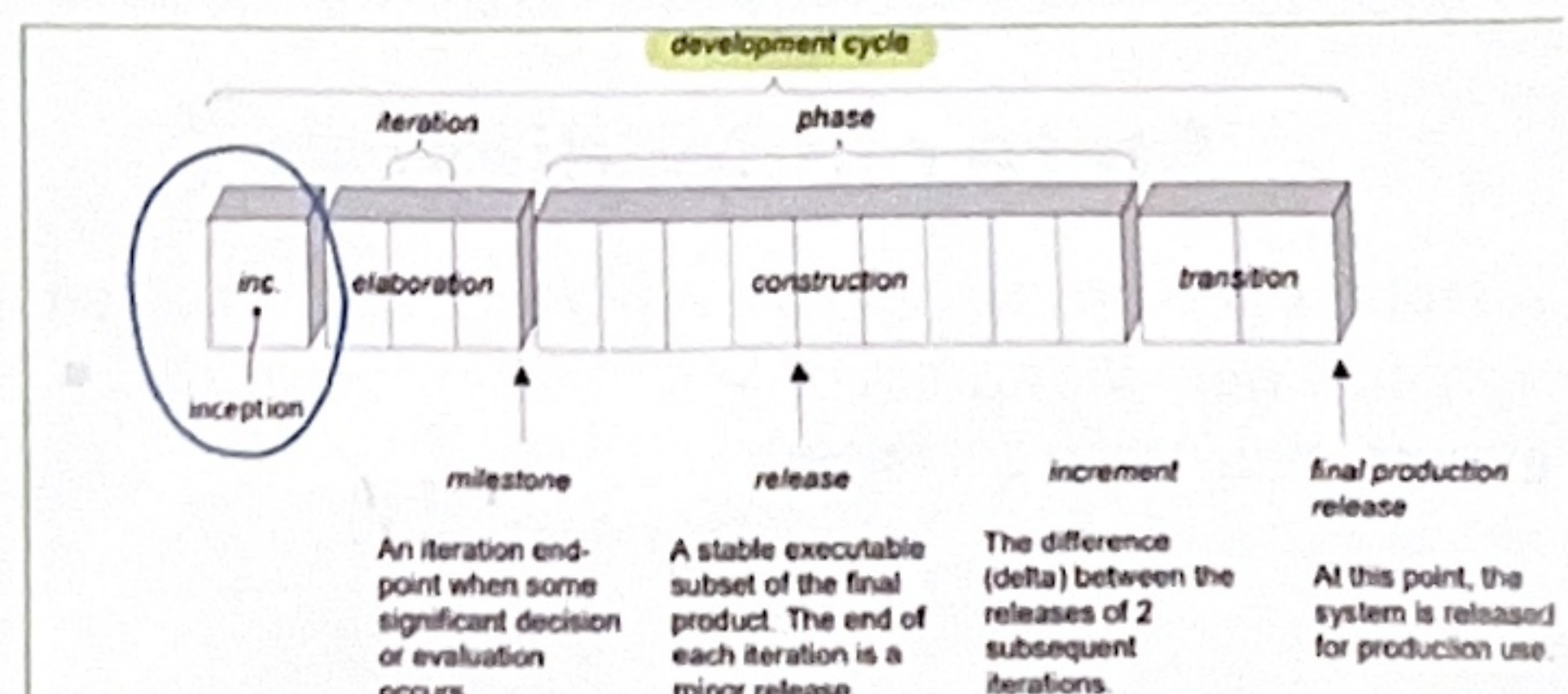
an iterative & incremental software development process

Inception

Inception is the initial short step to establish a common vision and basic scope for the project. The inception is not the time do all requirements or create believable estimates or plans. That happens during elaboration. Key task is name of use case models & 10% use case model in detail

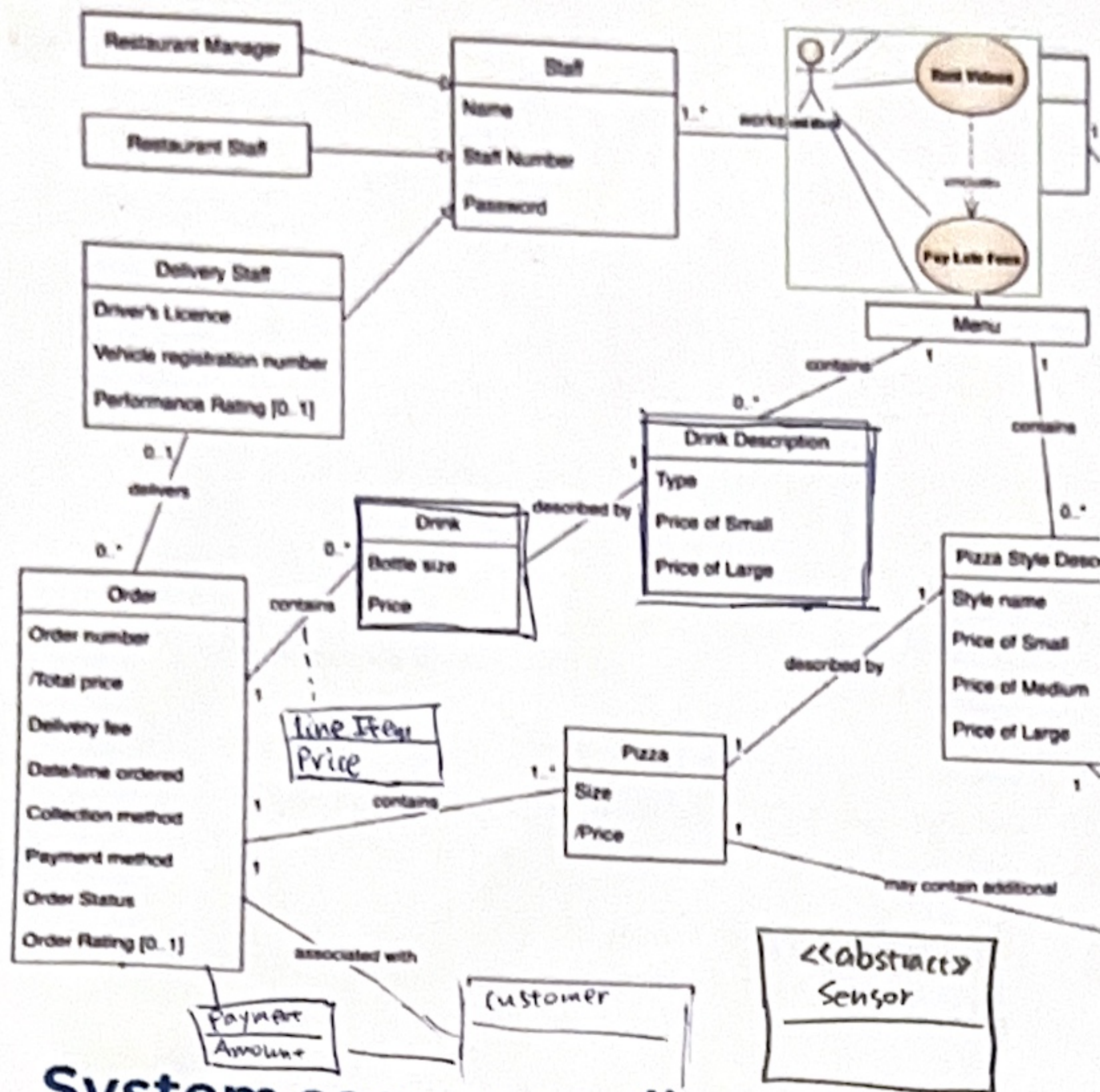
Elaboration

Elaboration is the initial series of iterations for building the core architecture, resolving the high-risk elements, defining most requirements, and estimating the overall schedule and resources. Elaboration is not a do-the-whole-design phase or a phase when the models are fully developed in preparation for implementation in the construction step. Key tasks are 80% reliably defined: domain model/design model/software architecture document/data model/use-case story-boards/UI Prototypes



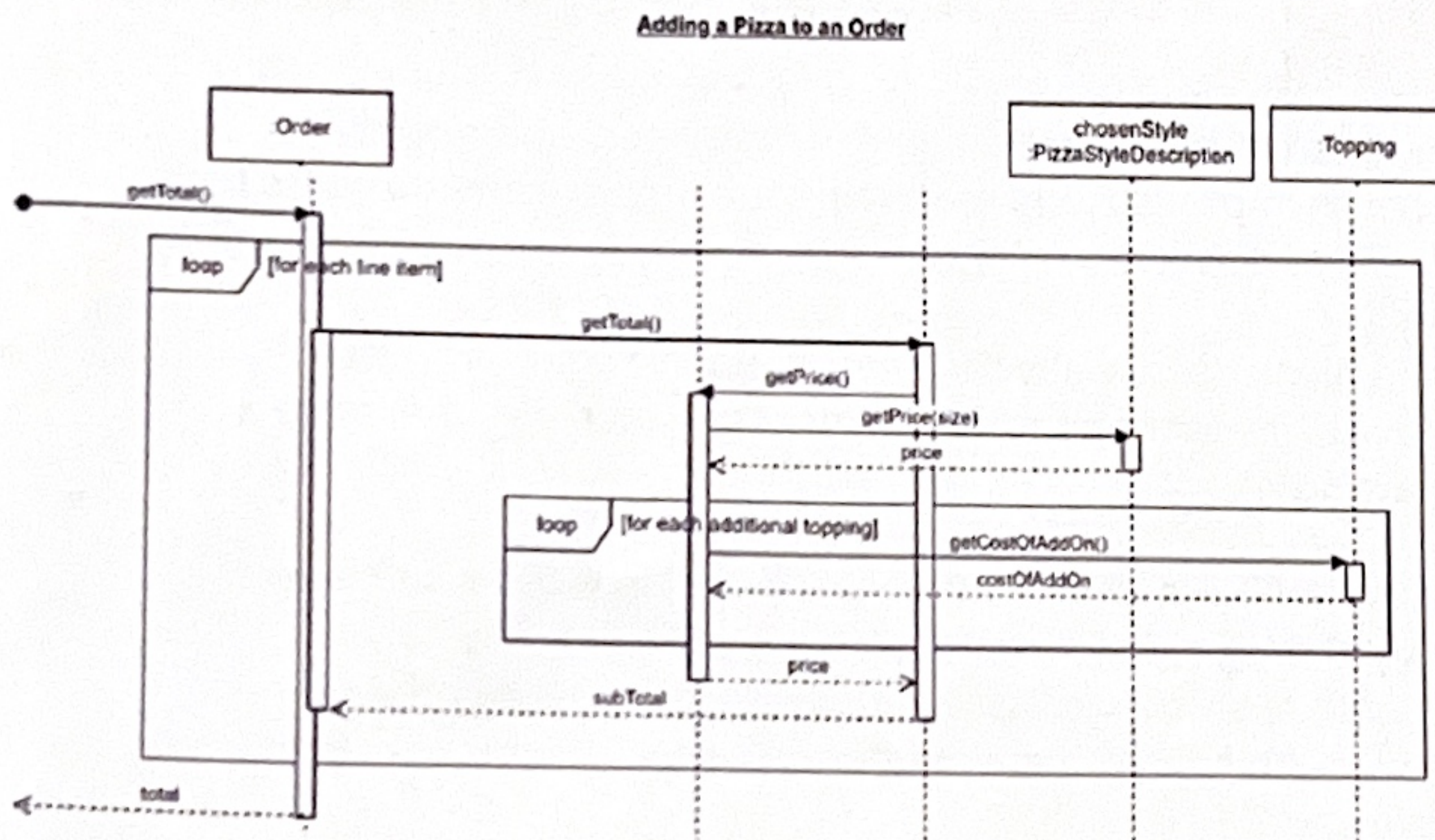
Domain class model

A domain class diagram provides a conceptual perspective to show conceptual classes, their attributes and associations with other classes



System sequence diagram

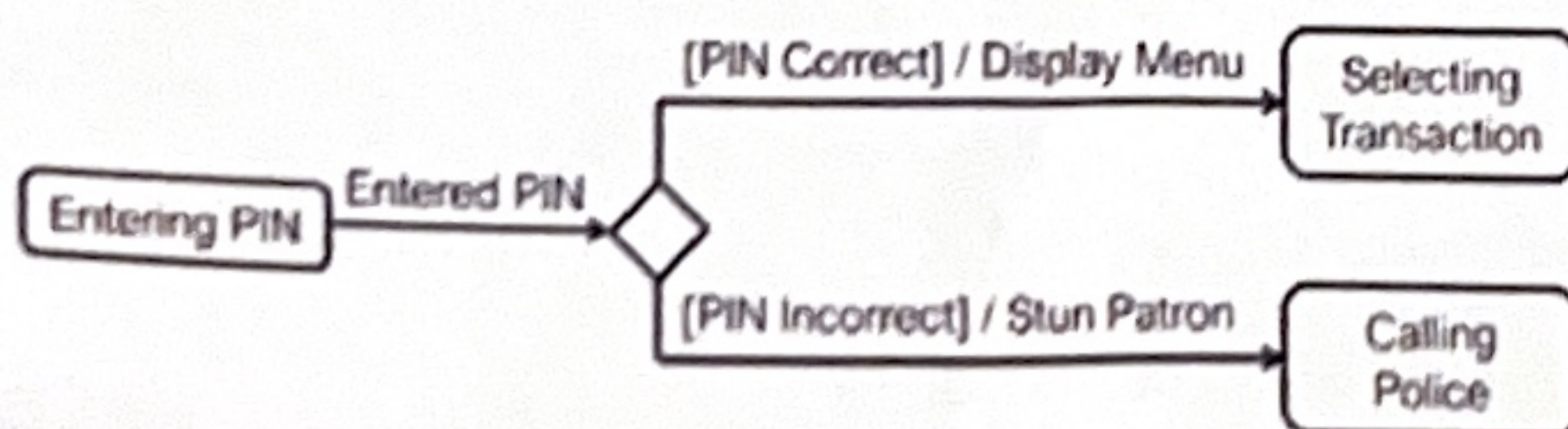
A visualisation of the system events that external actors generate, the order of the events, and possible inter-system events. One SSD is for one particular scenario of a use case. SSDs indicate events which design needs to handle and include associated information with reference to domain model.



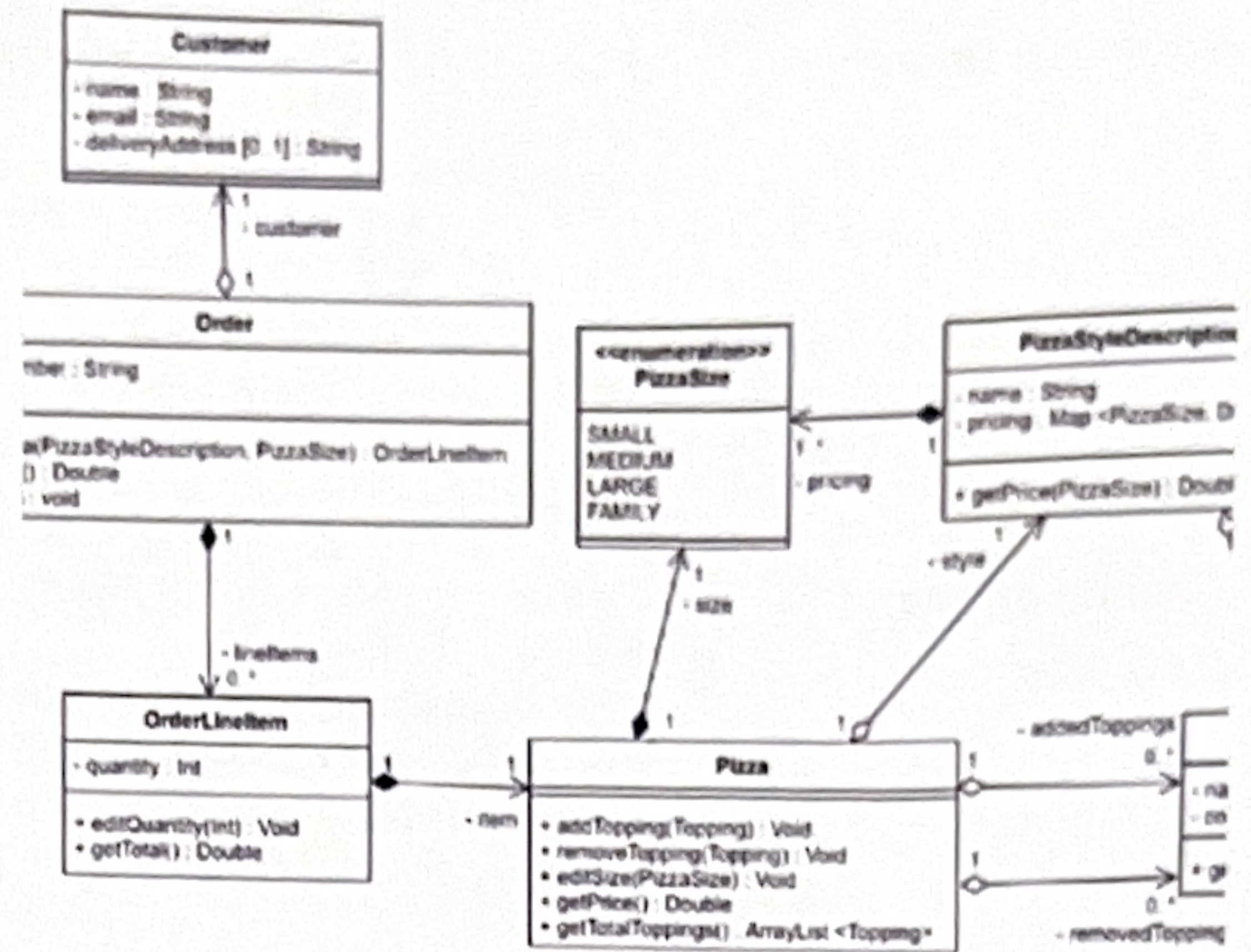
State model diagram

A state machine is a behavior model that captures the dynamic behavior of an object in terms of states, events, and state transitions.

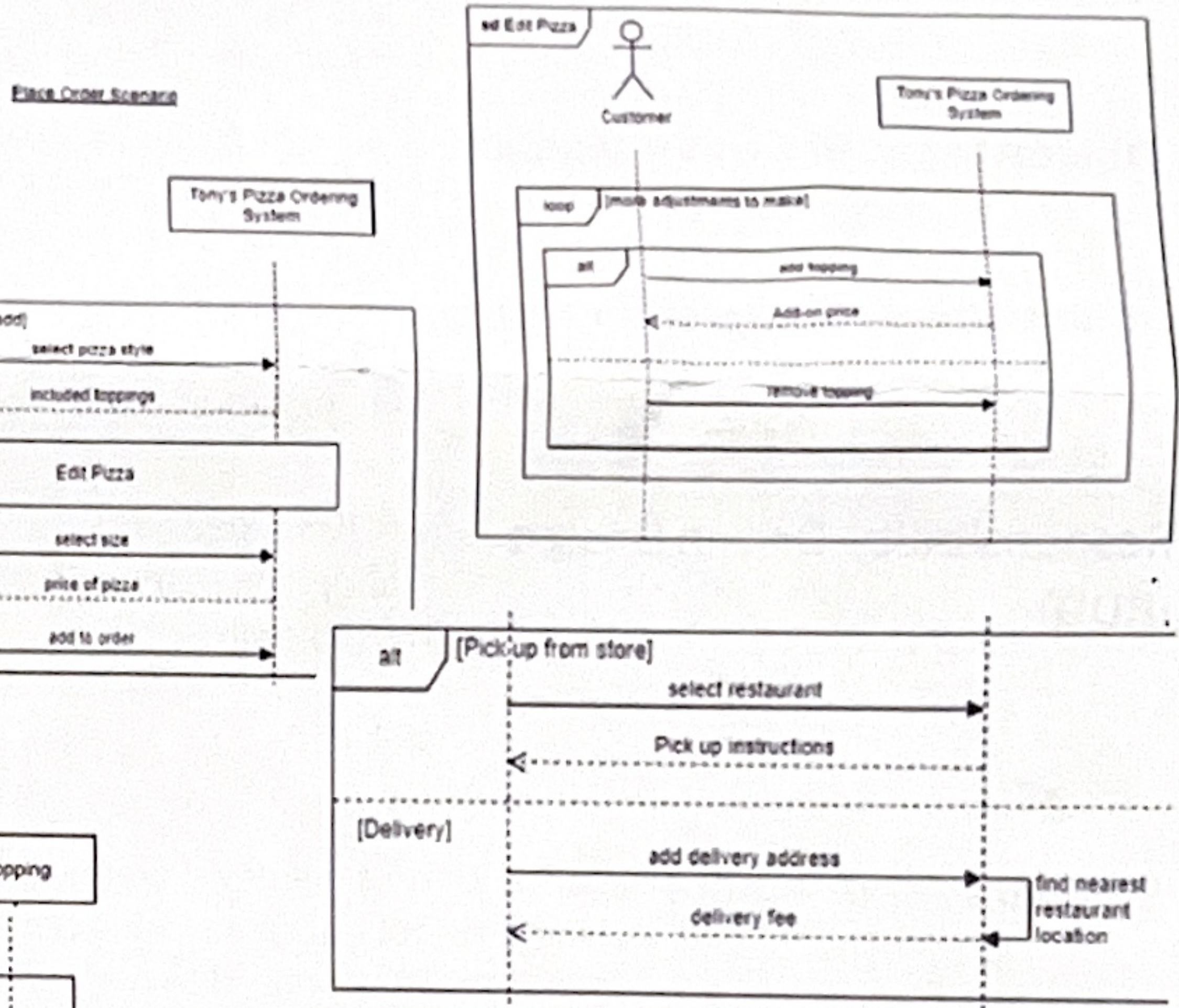
+ trigger [guard] / action



Design class model

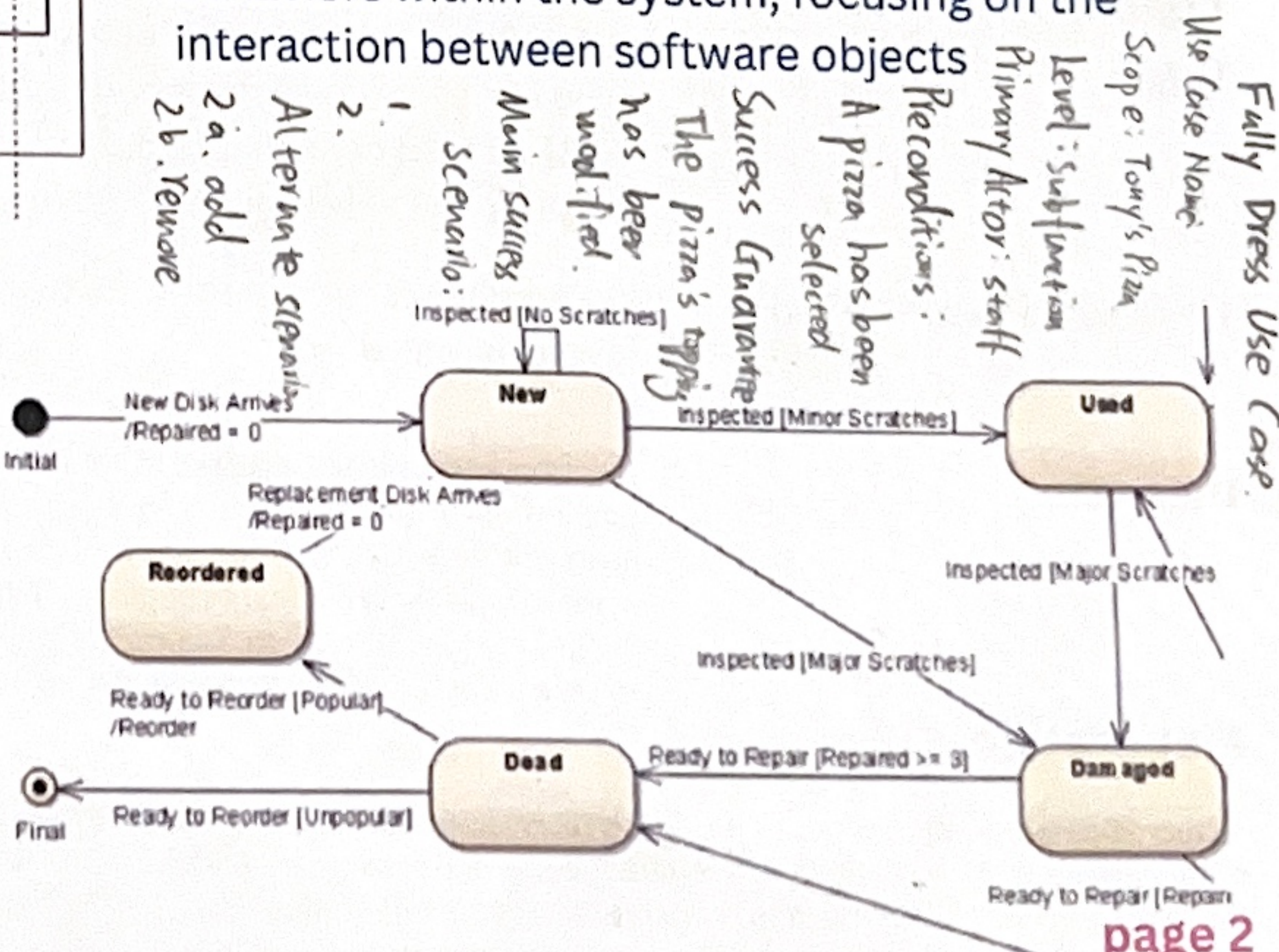


Note: filled rectangle is composition means sub-object's existence is dependent on the other
unfilled rectangle means one contains another



Design sequence diagram

Design Sequence Diagram illustrates the behaviors within the system, focusing on the interaction between software objects



Creator

Problem: Who should be responsible for creating a new instance of class A? (Doing Responsibility)

Solution: If an object A contains or often use an object B, an object A should be responsible for creating an object B (Task X = Creation of B)

Information expert

Problem: What is a general principle of assigning responsibilities to objects?

Solution: Assign the responsibility to object X if X has the information to fulfill that responsibility

Low coupling

Problem: How to support low dependency, low change impact, and increased reuse?

Solution: Assign responsibilities to avoid unnecessary coupling and to keep coupling low. Low Coupling minimizes dependencies, making the system more maintainable, and the code more reusable.

High Cohesion

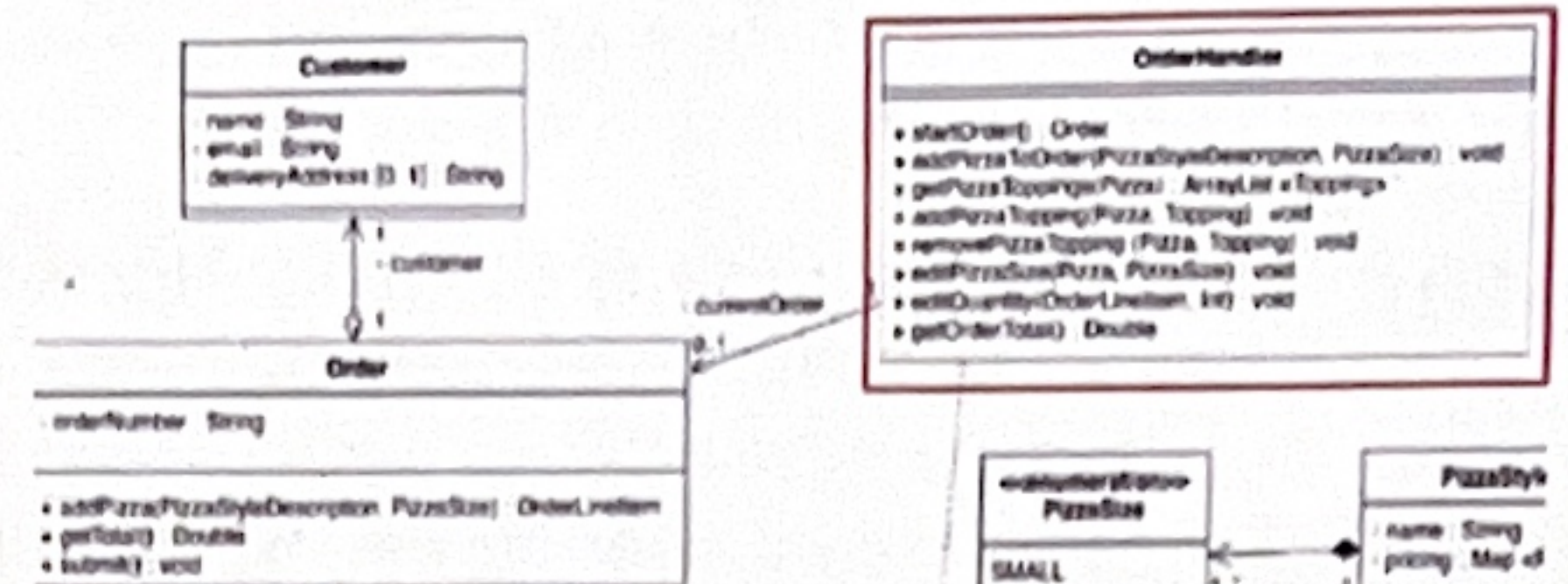
Problem: How to keep objects focused, understandable, and manageable, and as a side effect, support Low Coupling?

Solution: Assign responsibilities to avoid unnecessarily reducing cohesion and to keep cohesion high. High cohesion will ease of comprehension of the design, simplify maintainability and enhancement.

Controller

Problem: How to control and coordinate activities in the system, how to initiate actions in other objects?

Solution: create a handler object



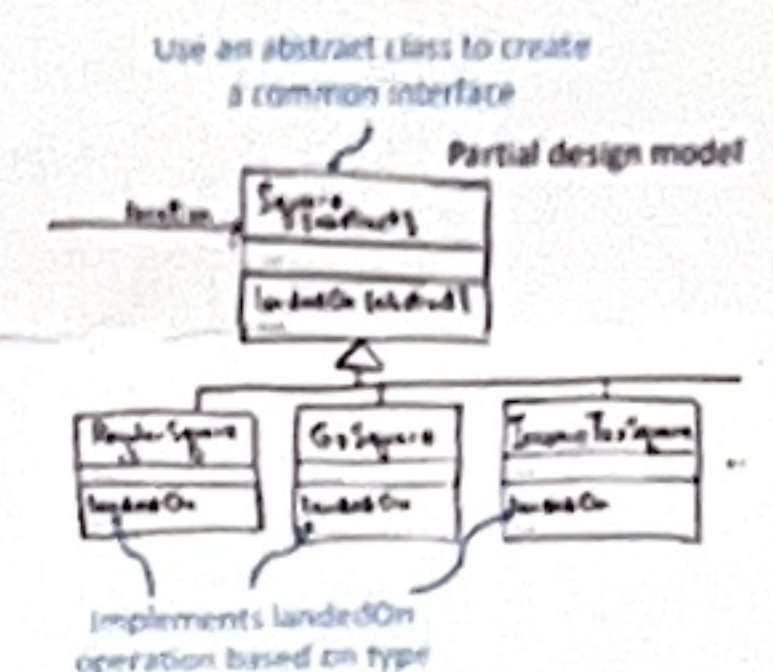
Polymorphism

Problem 1: How to handle alternatives based on type?

Problem 2: How to create pluggable software components?

– E.g., ClientServer relationship: How to replace Server component without affecting Client?

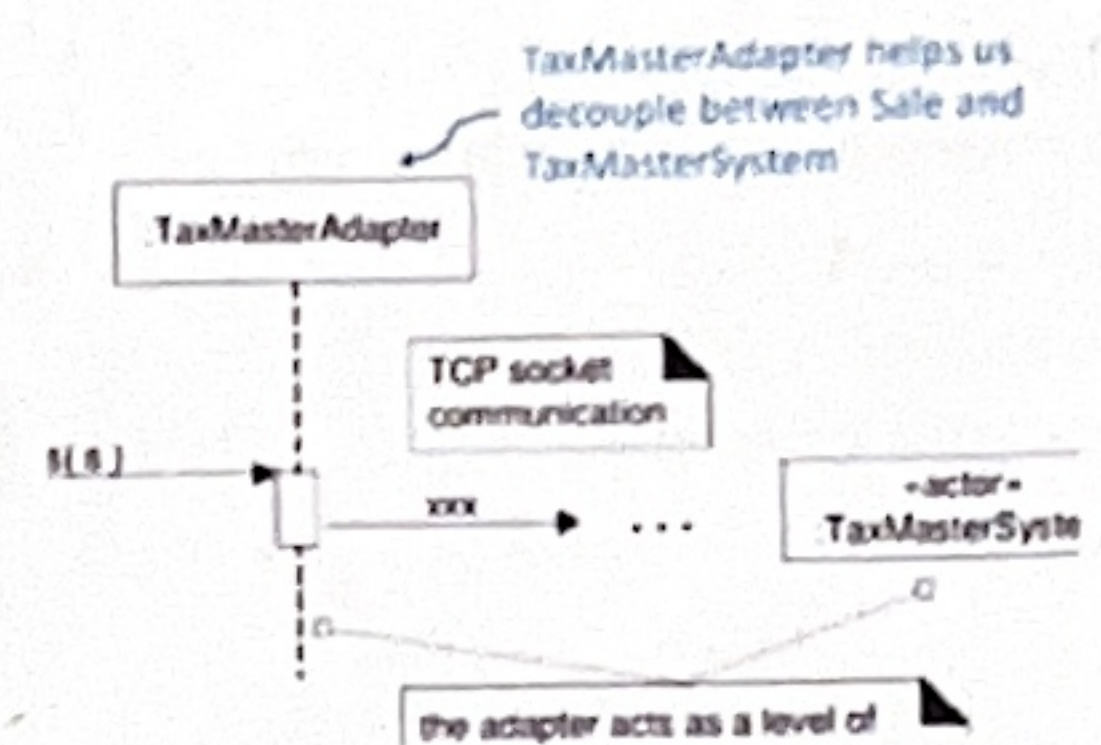
Solution: When related alternatives or behaviours vary by type (class), assign responsibility for the behaviour



Indirection

Problem: How to avoid direct coupling between two (or more) things? – How to decouple objects so that low coupling is supported, and reuse potential remains higher?

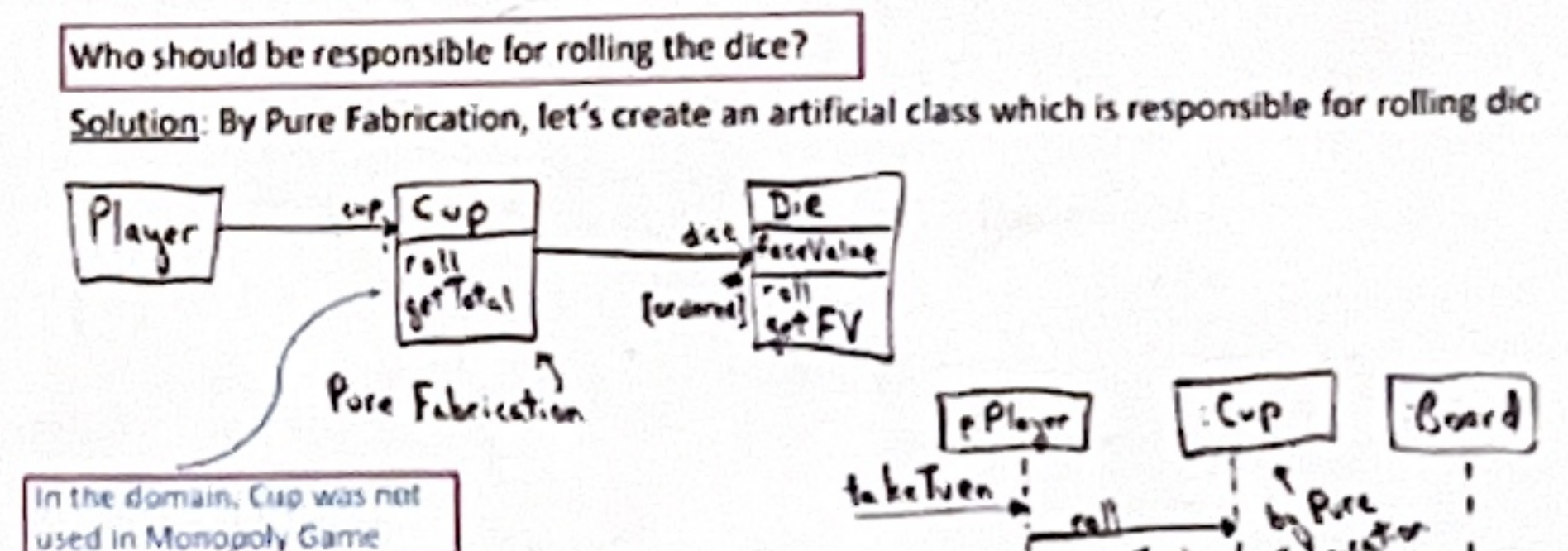
Solution: Assign the responsibility to an intermediate object to mediate between other components or services so that they are not directly coupled. – The intermediary creates an indirection between the other components.



Pure fabrication

Problem: Which object should have responsibility when you do not want to violate High Cohesion and Low Coupling, but solutions offered by other patterns (e.g., Expert) are not appropriate?

Solution: Assign a highly cohesive set of responsibilities to an artificial or convenience class that does not represent a problem domain concept



Protected variations

Problem: How to design objects, subsystems, and systems so that the variations in these elements does not have an undesirable impact on other elements?

Solution: Identify points of known or predicted variation or instability;

– Assign responsibilities to create a stable interface* around them

*Interface: a means of access (not only a programming language interface or Java interface)

Singleton

Q: Exactly one instance of a class is allowed –it is a “singleton.” Objects need a global and single point of access

A: Define a static method of the class that returns the singleton

```
1 class Singleton {
2     private static Singleton _instance = null;
3     private Singleton() {
4         //fill in the blank
5     }
6
7     public static Singleton getInstance() {
8         if ( _instance == null )
9             _instance = new Singleton();
10        return _instance;
11    }
12
13    public void otherOperations() { }
```


Facade

Q: How to design a common unified interface for a disparate set of implementations or interfaces such as within a subsystem?
A: Create a Pure Fabrication object called a Façade object to define a single point of contact to the subsystem. Facades are often accessed via Singleton.



ShapeDecorator.java

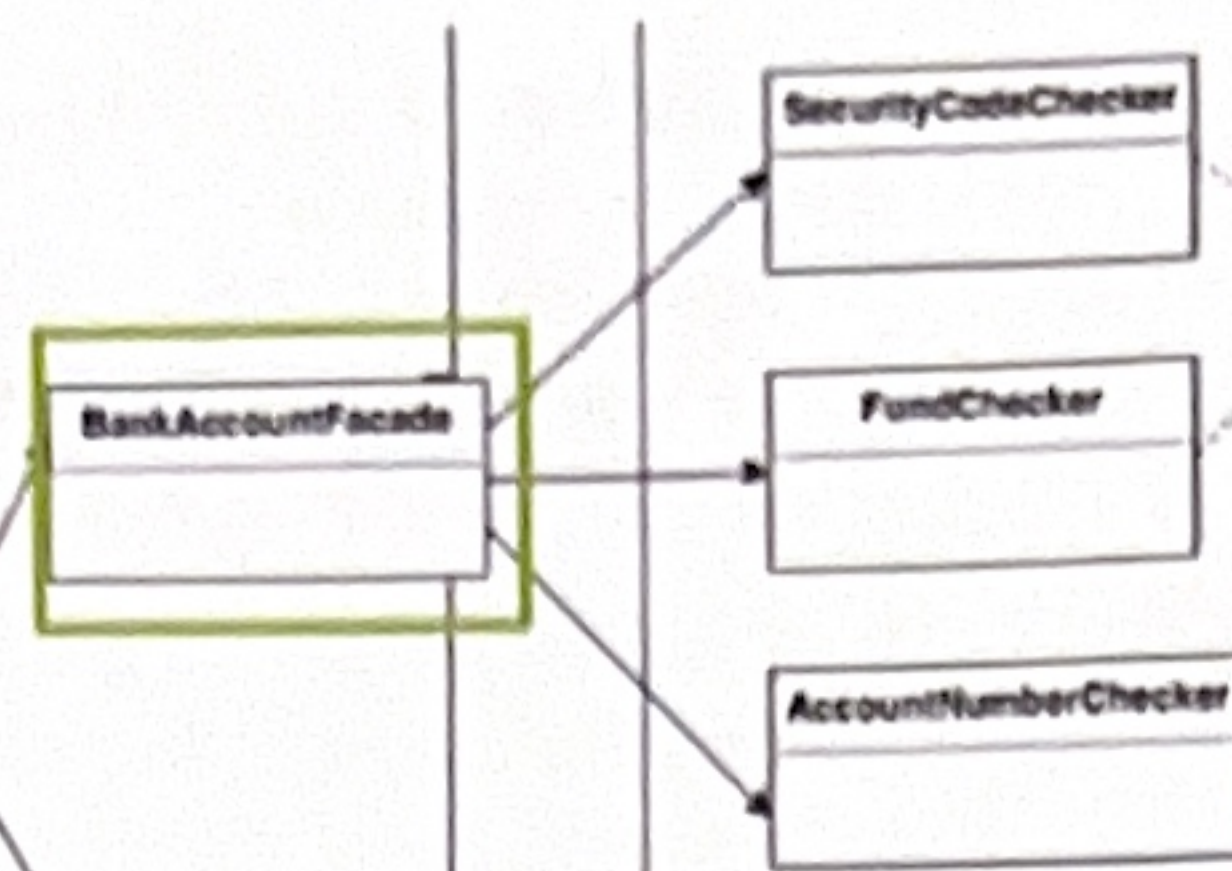
```

public abstract class ShapeDecorator implements Shape {
    protected Shape decoratedShape;

    public ShapeDecorator(Shape decoratedShape) {
        this.decoratedShape = decoratedShape;
    }

    public void draw() {
        decoratedShape.draw();
    }
}

```



Decorator

Definition: a structural design pattern that dynamically add behaviour or state to individual objects at run time without changing interface, and is done that by defining an interface that uses recursive composition to allow an unlimited number of decorator "layers" to be added to each core object.

```

abstract class JuiceMaker {
    public final Juice makeJuice() {
        Juice juice = createJuice();
        setJuice(juice);

        setJuiceType();
        setWater();
        setFruit();
        setSugar();

        return juice;
    }
}

```

despite the complexity and difference in asse abstract class but all the methods it calls are define

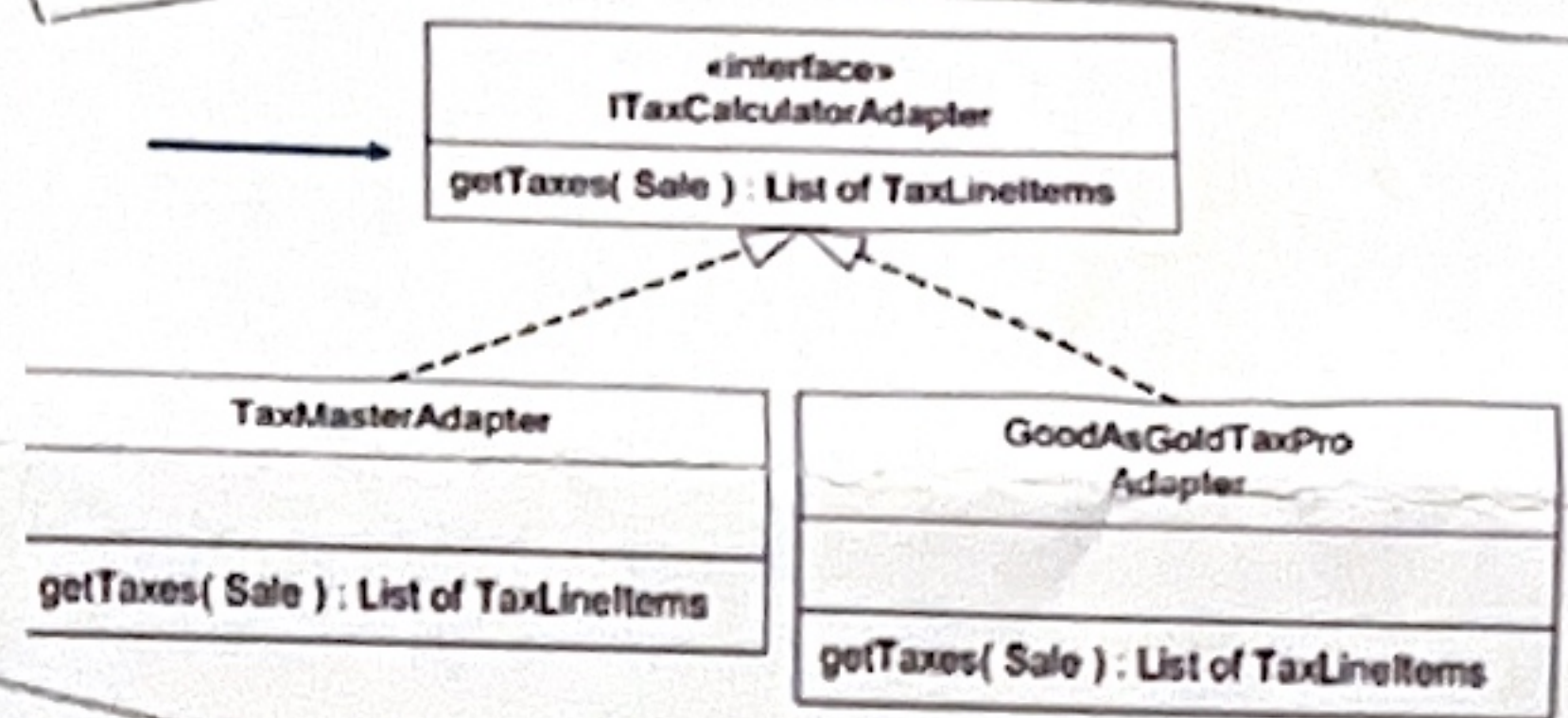


Builder

Definition: a creational pattern which uses domain knowledge to simplify the creation of a complex object by use the same construction process for different representation (all type of juices are juice).

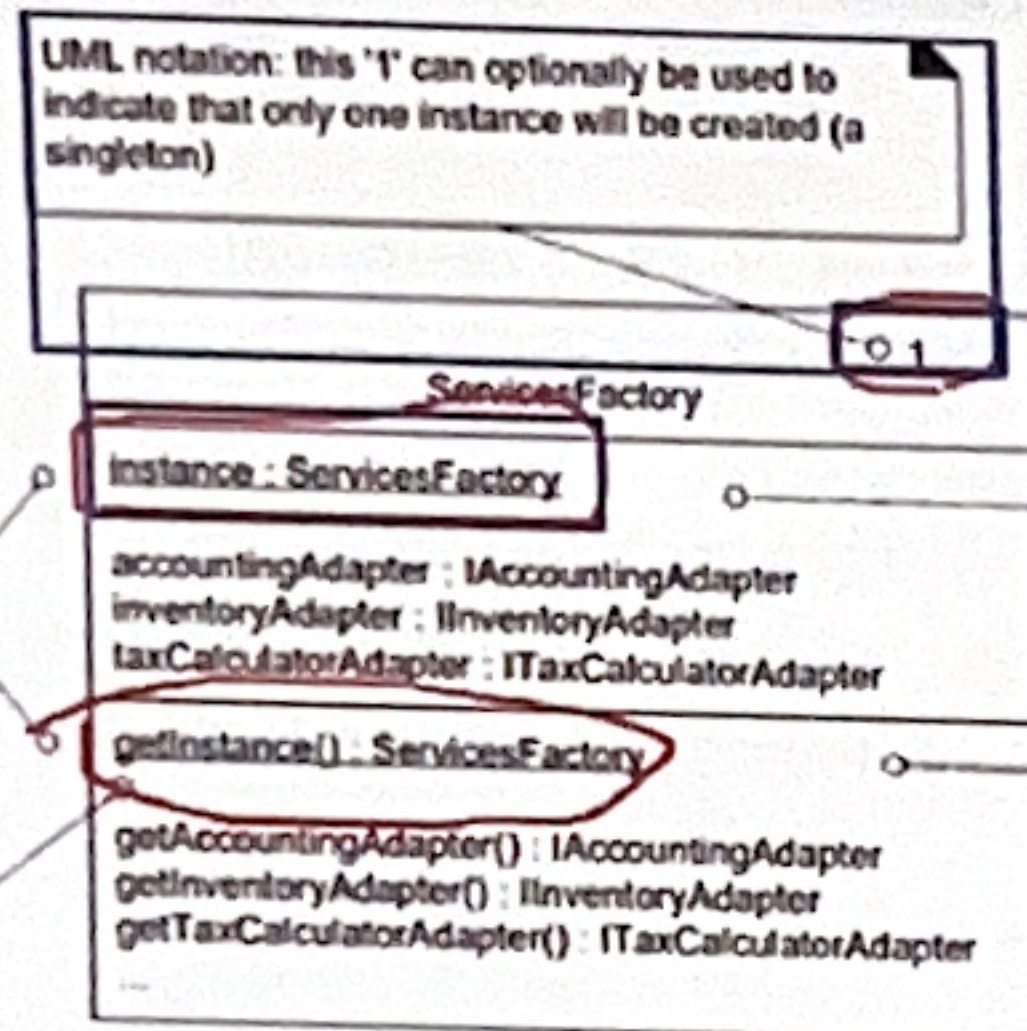
Adapter

Definition: a design pattern that allows objects with incompatible interfaces to work together. It acts as a bridge between two incompatible interfaces by converting the interface of a class into another interface that a client expects. This pattern is achieved in code by defining a java interface, and have different incompatible objects to implement that java interface, so that these objects look like they act with the same behaviours.

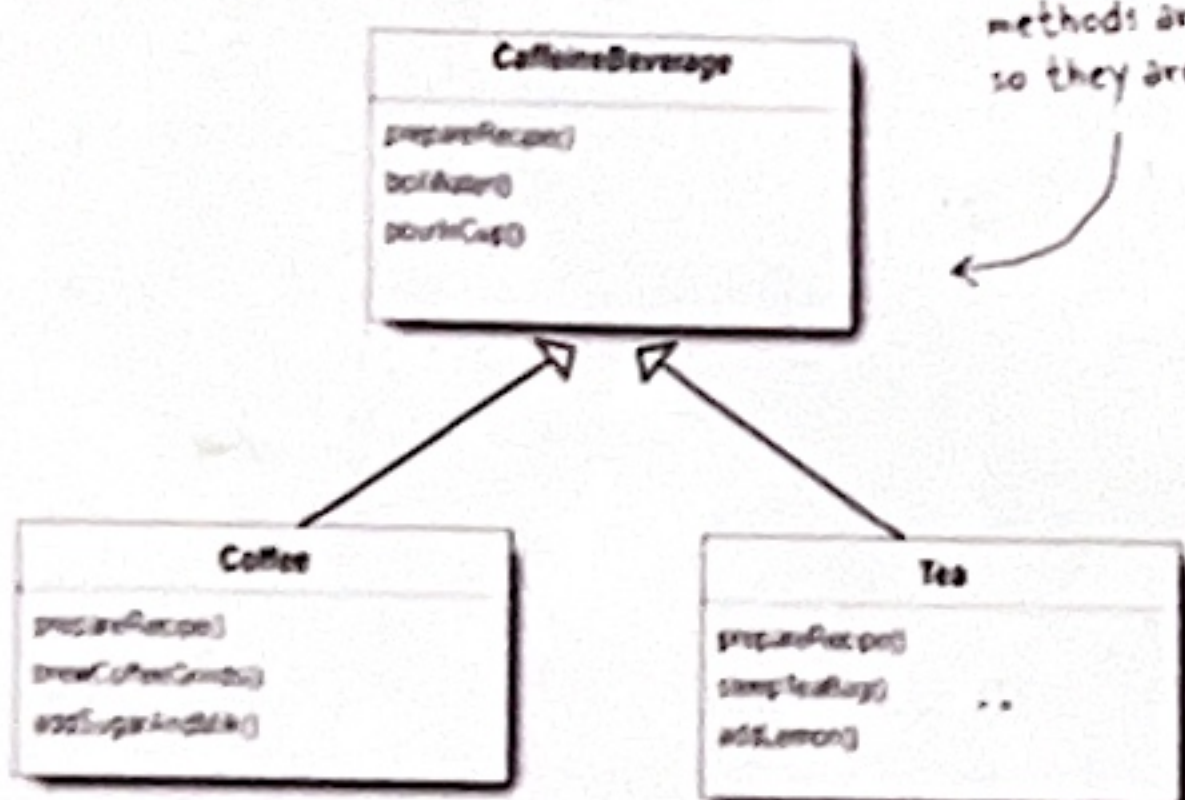


Factory

a design pattern that propose a pure fabrication object, a factory class (usually a Singleton) to handle complex creation logic.



The boilWater() and pourInCup() methods are shared by both subclasses, so they are defined in the superclass.



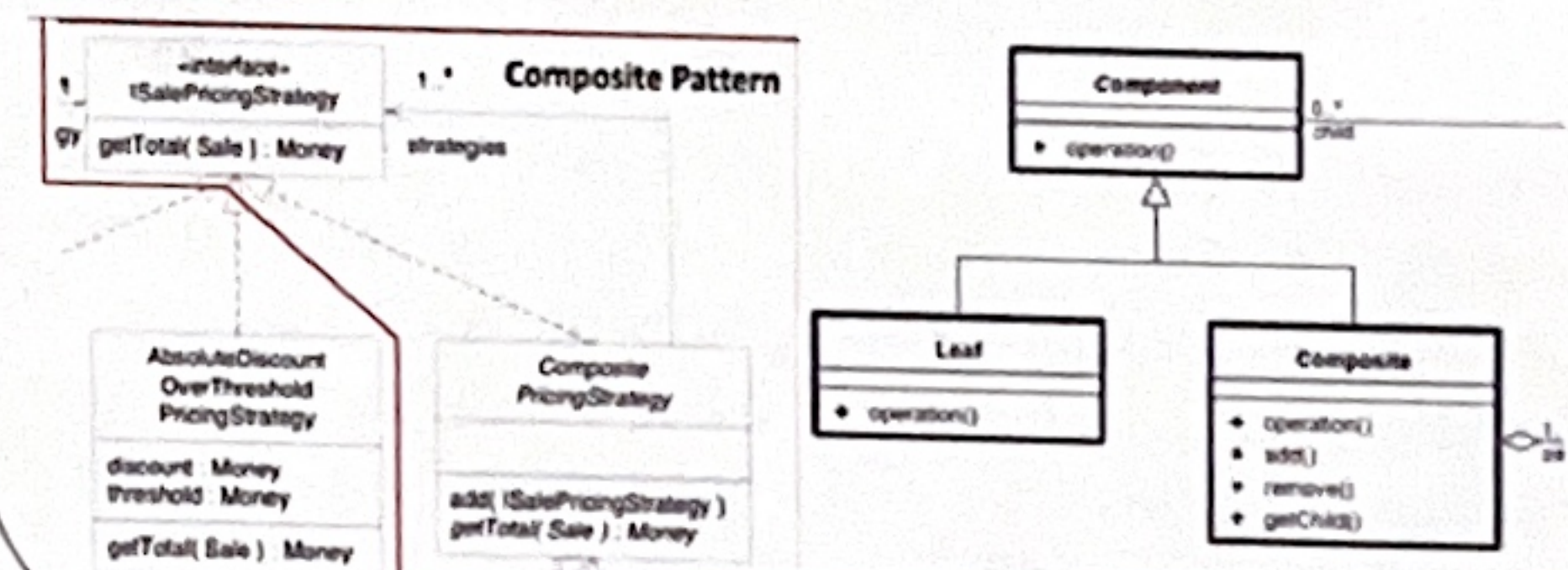
Template

a behavioral design pattern that defines the skeleton of an algorithm in a superclass, but allows subclasses to override specific steps of the algorithm without changing its overall structure

Composite

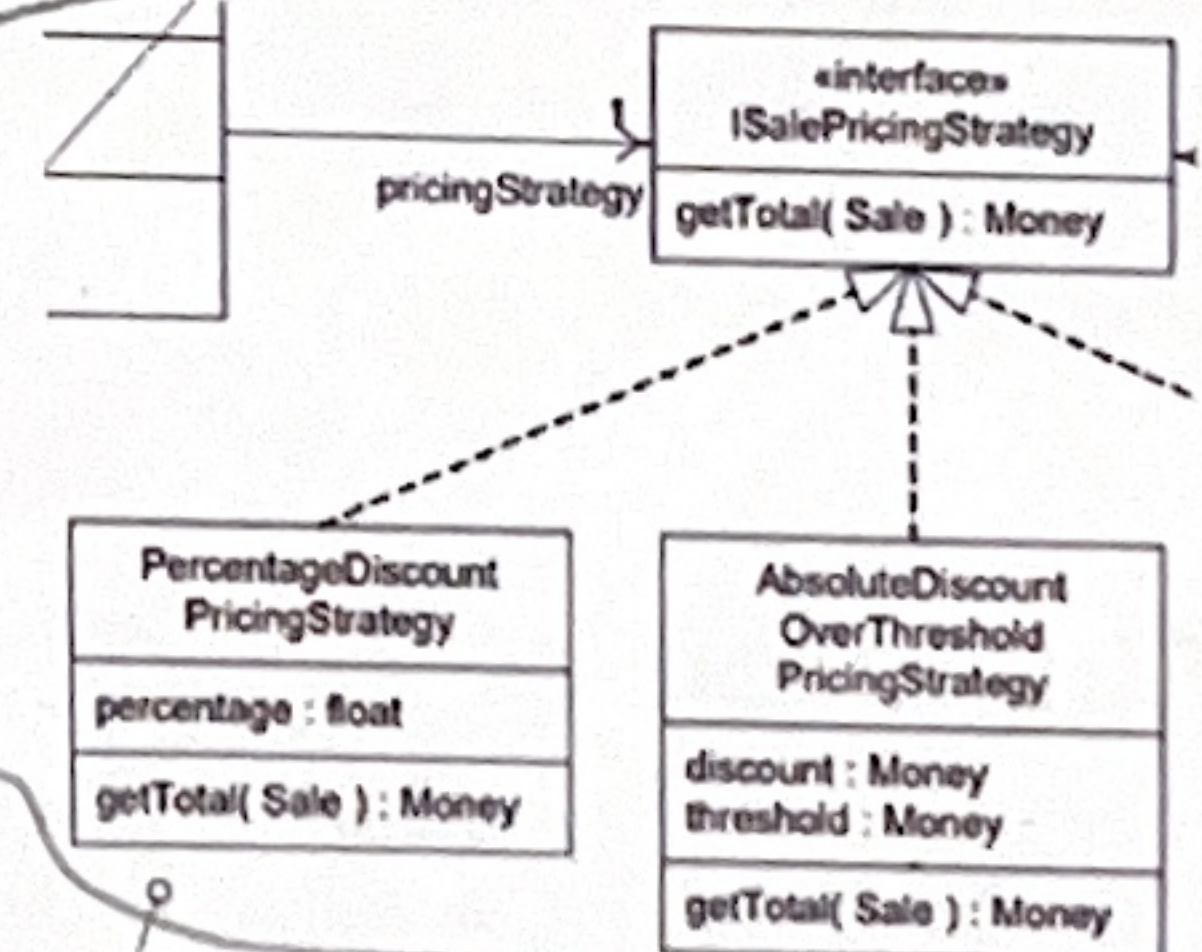
Q: How to treat a group or composition structure of objects the same way (polymorphically) as a non-composite (atomic) object?

A: Define classes for composite and atomic objects so that they implement the same interface. Treat individual objects and compositions (combinations of objects) uniformly.



Strategy

Q: How to design for varying, but related, algorithms, strategies or policies?
A: Define each algorithm/policy/strategy in a separate class, with a common interface. Enables selecting an algorithm's behavior at runtime.



Observer

a behavioral design pattern that define a "subscriber" or "listener" interface.

In this pattern, one object (the subject) maintains a list of its dependent objects (observers), and notifies them automatically whenever there is a change in its state

